CS24307 — THEORY (3.0 CR) FULL COURSE MASTER BIT MESRA | NEP SCHEME

Artificial Intelligence (CS24307) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PROBLEM BANK

Artificial Intelligence (CS24307)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

📚 Complete Course Structure & Algorithmic Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Intelligent Agents & Problem Solving	PEAS Description, Environment Types, State Space Formulation, Uninformed Search (BFS, DFS, DLS, IDDFS, UCS)
Module II	Informed Search & Game AI	Greedy Best-First, A* Search (Admissibility & Consistency Proofs), IDA*, Hill Climbing, Simulated Annealing, Minimax, Alpha-Beta Pruning, CSP
Module III	Knowledge & Logic Resolution	Propositional Logic (PL), First-Order Logic (FOL), CNF Conversion, Forward & Backward Chaining, Resolution Refutation
Module IV	Planning & Probabilistic Reasoning	STRIPS, PDDL Action Schemas, Planning Graphs, Bayesian Belief Networks (BBN), Conditional Independence, d-separation
Module V	Learning & Neural Networks	Inductive Learning, Decision Tree (ID3 Information Gain), Perceptrons, Multi-Layer Perceptrons (MLP), Backpropagation

🎯 Exam Preparation & High-Yield Strategy

This publication-grade master book consolidates all 5 modules with formal search completeness & optimality proofs, KaTeX-rendered logic formulations, game tree pruning traces, and model answers to BIT Mesra end-semester examination questions.

Module I: Intelligent Agents & PEAS — Topics Covered

1. Definitions of AI & Turing Test
2. Concept of Rationality vs. Omniscience
3. PEAS Framework (Performance, Environment, Actuators, Sensors)
4. Environment Taxonomies (7 Dimensions)
5. Agent Structures: Reflex, Goal, Utility & Learning Agents

1. PEAS Framework Formulation

Agent Type	Performance Measure (P)	Environment (E)	Actuators (A)	Sensors (S)
Automated Taxi Driver	Safety, speed, comfort, maximum profit, legal compliance	Roads, traffic, pedestrians, weather conditions	Steering wheel, accelerator, brake, horn, display	Cameras, LiDAR, radar, GPS, speedometer, engine sensors
Medical Diagnosis System	Patient health recovery, minimized cost, zero misdiagnosis	Patient, hospital database, laboratory staff	Screen displays, prescription generation, referral alerts	Keyboard input of symptoms, patient test results

2. Environment Dimensions Taxonomy

- **Fully Observable vs. Partially Observable:** Agent sensors give complete access to world state vs. noisy/occluded states.
- **Deterministic vs. Stochastic:** Next state is completely determined by current state and action vs. randomness.
- **Episodic vs. Sequential:** Current decision does not affect future episodes vs. current actions dictate future rewards.
- **Static vs. Dynamic:** Environment does not change while agent is deliberating vs. continuously evolving.
- **Discrete vs. Continuous:** Finite number of states/actions (Chess) vs. continuous state/time (Driving).

Module II: Problem Solving by Search & Game AI — Topics Covered

1. Uninformed Search: BFS, DFS, UCS, IDS
2. Heuristic Informed Search: Greedy & A*
3. Admissibility and Consistency of Heuristics
4. Local Search: Hill Climbing & Simulated Annealing
5. Adversarial Search: Minimax & Alpha-Beta Pruning

1. A^* Search Algorithm & Heuristic Optimality

A^* evaluates nodes by: $f(n) = g(n) + h(n)$, where $g(n)$ is the true path cost from start to n, and $h(n)$ is estimated cost from n to goal.

Theorems of A^* Optimality

Admissibility ($h(n) \leq h^*(n)$): A heuristic is admissible if it never overestimates the true cost to reach the goal. Admissibility guarantees that A^* tree search is optimal.
Consistency (Monotonicity): $h(n) \leq c(n, a, n') + h(n')$. Consistency guarantees that A^* graph search is optimal without reopening closed nodes.

2. Minimax with Alpha-Beta Pruning

α is the highest value found so far for MAX; β is the lowest value found so far for MIN. **Pruning Rule:** Whenever $\alpha \geq \beta$, prune the remaining subtrees below the current node.

Module III: Knowledge Representation & Logic — Topics Covered

1. Propositional Logic Syntax & Truth Tables
2. Inference Rules & Forward/Backward Chaining
3. Propositional Resolution Refutation
4. First-Order Logic (FOL) & Quantifiers
5. Unification (MGU) & Skolemization

1. First-Order Logic Resolution Refutation

To prove $\mathrm{KB} \models \alpha$, we add $\neg\alpha$ to KB and derive the empty clause ($\square$) using 7 systematic CNF transformation steps:

1. Eliminate $\iff$ and $\implies$.
2. Push $\neg$ inward ($\neg\forall x P(x) \equiv \exists x \neg P(x)$).
3. Standardize variables (distinct names per quantifier).
4. **Skolemization:** Replace existential quantifiers ($\exists$) with Skolem constants or functions of outer universal variables.
5. Drop universal quantifiers ($\forall$).
6. Convert to CNF using $\vee$ distribution over $\wedge$.
7. Resolve complementary literals using **Most General Unifier (MGU)**.

Module IV: Planning & Probabilistic Reasoning — Topics Covered

1. STRIPS Planning Representation & PDDL
2. Goal Stack Planning & Sussman Anomaly
3. Axioms of Probability & Bayes' Rule
4. Bayesian Belief Networks & Joint Factorization
5. D-Separation & Exact Probabilistic Inference

1. Bayesian Network Joint Probability Factorization

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \mathrm{Parents}(X_i))$$

A Bayesian Network is a Directed Acyclic Graph (DAG) whose nodes represent random variables and directed edges represent conditional dependencies.

Module V: Machine Learning Foundations — Topics Covered

1. Inductive Learning & Decision Trees (ID3)
2. Entropy & Information Gain Formulations
3. Single-Layer Perceptron Learning Rule
4. Multi-Layer Perceptron & Backpropagation
5. Bias-Variance Tradeoff & Regularization

1. Decision Tree Induction: Entropy & Information Gain

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

$$\mathrm{Gain}(S, A) = H(S) - \sum_{v \in \mathrm{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

BIT Mesra Exam Question (10 Marks)

Problem: Explain Backpropagation algorithm for training a Multi-Layer Perceptron with gradient descent derivations.
Solution: Forward pass computes net activations $z_j = \sum w_{ij} x_i$ and outputs $a_j = \sigma(z_j)$. Backward pass propagates error $\delta_k = (y_k - a_k)\sigma'(z_k)$ to hidden layers $\delta_j = \sigma'(z_j) \sum w_{jk}\delta_k$, updating weights via $w_{ij} \leftarrow w_{ij} + \eta \delta_j x_i$.